Class 6: R functions

Joshua Chun (A17812847)

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function).

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either **"required"** or **"optional"**. The later have fall-back default values that will be used if the user does not specify them.

```
add <- function(x, y=0) {
  x + y
}
```

> Q1a. Your first version of the function should add two input numbers together. For example, add(4,7) should return 11. Can we use our new function:

```
add(4,7)
```

```
[1] 11
```

```
add(10)
```

```
[1] 10
```

Q1b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). [1 pt]

```
add <- function(x, y= 0) {
  sum(x + y)
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4)

```
add <- function(...) {
  sum(...)
}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

```
add(1,2,3,-4)
```

```
[1] 2
```

We can explicityly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

A generate_dna() function

A useful function here is the "base R" `sample()` function:

```
sample(1:5, size=6, replace = TRUE)
```

```
[1] 3 3 4 4 2 5
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G", and "T"...

```
sample(x=c("A", "C", "G", "T"), size=10, replace = TRUE)
```

```
 [1] "T" "G" "A" "T" "T" "G" "T" "T" "G" "C"
```

> Q2a. Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C". [1 pt]

```
generate_dna <- function(len) {
  sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
}
```

```
generate_dna(100)
```

```
  [1] "A" "C" "G" "A" "C" "A" "C" "A" "C" "G" "T" "G" "A" "C" "G" "T" "A" "A"
 [19] "C" "T" "A" "T" "G" "C" "C" "G" "A" "A" "C" "A" "A" "C" "G" "G" "A" "C"
 [37] "A" "A" "T" "A" "A" "C" "C" "T" "T" "G" "G" "G" "C" "A" "C" "T" "A" "A"
 [55] "C" "T" "G" "T" "C" "G" "A" "C" "A" "T" "T" "A" "G" "G" "A" "A" "C" "T"
 [73] "A" "G" "A" "A" "C" "G" "T" "T" "A" "T" "A" "G" "A" "G" "G" "G" "T" "G"
 [91] "G" "G" "C" "C" "G" "G" "T" "G" "G" "G"
```

> Q2b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG". [1 pt]

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
  if(single.element) {
    ans <- (paste(ans, collapse = ""))
  }
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna(single.element = TRUE)
```

```
[1] "TTTAGGCCAA"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
  if(single.element) {
    ans <- (paste(ans, collapse = ""))
  }
  cat(paste(">len", len, "\n", sep=""))
  cat(ans)
  cat("\n")
  return(ans)
}
```

```
x <- generate_dna(44)
```

```
>len44
ATTTTGTCGTCATACGAATGGTCGTAGATACTGACAGACCCGCA
```

Q3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
generate_protein <- function(len) {
  ans <- sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I",
          "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V"), size=len, replace = TRUE)

    ans <- cat((paste(ans, collapse = "")))


  return(ans)
}
```

```
generate_protein(6)
```

```
CIMWHL
```

```
NULL
```

Generate random protein sequences of length 6 to 13

> Q4. Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 sto 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

```
for(l in 6:13) {
  cat(">", l, "\n", sep="")
  cat(generate_protein(l), "\n")
}
```

```
>6
YAAMYG
>7
AKRHHKL
>8
HQLPDRKE
>9
FLAMARGTQ
>10
PNCPSEADAC
>11
ADTGFFFARHN
```

```
>12
GTNKIEHVMHRE
>13
VNEEAECEDYMHE
```

Are Our peptides "unique in nature"?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	88	Y
9	89	100	Y
10	100	83	Y
11	100	82	Y
12	83	90	Y
13	85	72	Y

a. My randomly generated peptides start to look unique in nature at length 8, because that is the first length where there is no hit with both 100% identity and 100% coverage.

b. Very short peptides are often found in nr because the total number of possible sequences is still relatively small compared with the large number of short subsequences contained within known proteins. Since peptide sequence space grows as 20^L, the number of possibilities increases a lot with length, so by 8–13 amino acids the possible random sequences already exceed what has likely been sampled in the known protein universe, making exact matches much less likely.

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

Based on my Q5 results, the minimum length is likely around 8–10 amino acids, because this is where peptides begin to appear "unique". Very short peptides of around 6–7 amino acids exist frequently within many proteins because the total sequence space (20^L) is still relatively small, so they are highly likely to match "self" proteins. If the immune system presented such short peptides, it would risk failing to distinguish self from non-self, leading to either missed infections or autoimmune reactions. As peptide length increases, the sequence space expands

exponentially, making it far less likely that a foreign peptide matches anything in the host proteome. Therefore, using longer peptides improves the specificity of T-cell recognition and helps ensure a more accurate immune response.